# A2: Weight decay and phase transitions

Assignment handout

Training dynamics, a self-study course · written by Claude for Fan Pu

October 29, 2026

about 4.5 hours · CPU only · all six checks together run in about 25 seconds

## 1 Overview

Add weight decay to the anchor and it acquires a *phase transition*: below a critical strength the network learns a shrunken version of $y = x$, above it the learned function is exactly zero. This assignment finds every stationary point of the regularized anchor, shows that the transition at two layers is *continuous* and that convergence slows down without bound as one approaches it, then adds a third layer and watches the transition turn *discontinuous*: the learned product jumps to zero, and zero becomes a trap that small initializations fall into even at small weight decay. Last, finite step sizes carve oscillation and divergence regions out of the $(\gamma, \eta)$ plane, reproducing the diagram at the end of Ang's post. A1 supplied the conserved imbalance and the logistic learning curve, both reused here; A3 (edge of stability) builds on the step-size analysis of the last problem.

**What you will derive**

1. That weight decay balances the layers at every stationary point, for scalars, matrices and depth 3 (Problem (`balance`))
2. All stationary points of the two-layer loss, their Hessians and types, and the bifurcation at $\gamma = \frac{1}{2}$ (Problem (`stationary`))
3. The relaxation time near $\gamma = \frac{1}{2}$ with its exact constants, and the algebraic decay at $\gamma = \frac{1}{2}$ (Problem (`slowing`))
4. The depth-3 critical value $\gamma^*$, the jump in the learned product, and when zero is the global minimum (Problem (`depth3`))
5. The basin of the zero solution on the balanced ray and its size for small weight decay (Problem (`basins`))
6. The step-size stability boundary in the $(\gamma, \eta)$ plane, and the oscillation band on the balanced line (Problem (`finite-step`))

**What you will implement**

1. `stationary.py`, `slowing.py` in `wdecay/`: stationary points with types, learned product, relaxation time, time to converge (Problems (`stationary`), (`slowing`))
2. `depth3.py`, `basins.py`: the depth-3 stationary points, $\gamma^*$, $\gamma_c$, the endpoint of a balanced start (Problems (`depth3`), (`basins`))
3. `finite_step.py`: the largest stable step and the predicted phase of GD (Problem (`finite-step`))

**What you will run**

1. Six check scripts, one per problem, each comparing a simulation with your derivation
2. A short reading of Ziyin, Li and Meng on deep linear networks with weight decay (Problem (`ziyin`))

**Setup.** From the course root: `uv run pytest a2-weight-decay` (all 48 tests fail with `NotImplementedError` until you fill in the stubs). Checks run as `uv run python a2-weight-decay/scripts/check_<name>.py` and write PNGs and their section of `a2-weight-decay/out/results.json`; compare with `uv run python -m`

`tdyn.results a2-weight-decay/out/results.json a2-weight-decay/solutions/results.json`. Five checks have a `FORMULAS` block at the top where you type a derived formula. Worked solutions: `solutions/solutions.pdf`.

**Notation.** As in A0 and A1: $\theta = (a, b)$, residual $r = ab - s$ with target $s > 0$ ($s = 1$ unless stated), product $p = ab$, imbalance $c = a^2 - b^2$, step size $\eta$, weight decay $\gamma$. At depth 3, $\theta = (a, b, c)$ and $p = abc$ (there $c$ is the third weight, not the imbalance). $u$ is the common value of the weights on a *balanced* point $a = b(= c)$. $H = \nabla^2 L_\gamma$, with eigenvalues $\lambda_{\min} \le ... \le \lambda_{\max}$.

## 2 Weight decay and bifurcations

*Weight decay* adds $\gamma\|\theta\|^2$ to the loss. On the anchor,

$$L_\gamma(a, b) = \frac{1}{2}(ab - 1)^2 + \gamma(a^2 + b^2), \tag{1}$$

with gradient flow

$$\dot{a} = -br - 2\gamma a, \quad \dot{b} = -ar - 2\gamma b, \quad r = ab - 1. \tag{2}$$

In GD the extra term multiplies the weights by $1 - 2\eta\gamma$ every step, hence the name. The factor 2 in $2\gamma$ is this course's convention; it puts the critical value at $\gamma = \frac{1}{2}$, as in Ang's post [J. M. L. Ang, 2026, §4].

A *stationary point* (or critical point) is a $\theta^*$ with $\nabla L_\gamma(\theta^*) = 0$; it is a fixed point of the flow. A0 classified fixed points by the Jacobian of the flow, which here is $-H$: $\theta^*$ is a strict local *minimum* (stable) if every eigenvalue of $H(\theta^*)$ is positive, a *saddle* if there are both signs, a *maximum* if all are negative, and *degenerate* (the linearization says nothing) if one is zero. A *bifurcation* is a qualitative change in the set of fixed points or in their stability as a *control parameter* crosses a critical value; here the control parameter is $\gamma$, which the post compares to temperature in a material. An *order parameter* is a quantity that is zero on one side of the transition and nonzero on the other; ours is the learned product $p_\infty = \lim_{t\to\infty} ab$. The transition is *continuous* if $p_\infty$ goes to zero continuously at the critical value and *discontinuous* if it jumps.

Ziyin, Li and Meng find the global minima of deep linear networks with weight decay in closed form [L. Ziyin et al., 2022]. Their Theorem 1 says a network with one hidden layer learns nothing (global minimum at zero) iff $\|\mathbb{E}[xy]\|^2 \le \gamma_u\gamma_w$; in our convention ($2L_\gamma$ has $\gamma_u = \gamma_w = 2\gamma$ and $\mathbb{E}[xy] = s$) that is $\gamma \ge s/2$. Read their §3–4 for general widths and inputs. Note their "depth" counts *hidden* layers: our $f = abcx$ is their $D = 2$.

**Example (`ridge`): One layer has no transition**

Take a single weight, $f = wx$, $L_\gamma(w) = \frac{1}{2}(w - 1)^2 + \gamma w^2$ (ridge regression). $L_{\gamma'}(w) = (1 + 2\gamma)w - 1$, so the only stationary point is $w^* = 1/(1 + 2\gamma)$, a minimum with Hessian $1 + 2\gamma > 0$. The learned function shrinks smoothly, never reaches zero for finite $\gamma$ ($w^* = 1/2$ at $\gamma = 1/2$, $1/3$ at $\gamma = 1$), and gradient flow converges at rate $1 + 2\gamma \ge 1$ for every $\gamma$: no transition and no slowing down. Ziyin et al.'s Figure 1 (right) makes the same point for a linear regressor against a ResNet. The transitions below come from the *product* of layers.

**Problem (`balance`): Weight decay balances the layers** [Derive + Check, 25 min] **[you]**

(a) **[Derive]** For $L_\gamma = \ell(ab) + \gamma(a^2 + b^2)$ with any differentiable $\ell$, compute $a\,\partial_a L_\gamma - b\,\partial_b L_\gamma$ (A1 Problem (`noether`) did $\gamma = 0$). Conclude what every stationary point satisfies, and quote $\dot{c}$ along the flow from A1 Problem (`broken`) (a). **Deliverable**: the identity; the constraint on stationary points; $c(t)$.

(b) **[Derive]** For a two-layer linear network $W_2W_1$ with any loss $L(W_2W_1)$ plus $\gamma\left(\|W_1\|_F^2 + \|W_2\|_F^2\right)$, show that $D = W_1W_1^\top - W_2^\top W_2$ obeys a closed linear ODE and solve it. What does every stationary point satisfy, and what does the $(j, j)$ entry of that say about hidden neuron $j$? (This is Ziyin et al.'s Lemma 1, §3.1, from the flow instead of from optimality.) **Deliverable**: $D(t)$ (type it into `imbalance_at` in `scripts/check_balance.py`); the per-neuron statement. *Hint:* A1 Problem (`matrix-law`) (b) showed that the $\gamma$-free part of the gradient leaves $D$ fixed.

(c) **[Derive]** Repeat (a) for $f = abcx$ with $\gamma(a^2 + b^2 + c^2)$: which differences of squares decay, and what holds at a stationary point? **Deliverable**: the decay law for $a^2 - b^2$ and $b^2 - c^2$; the stationary constraint.

(d) **[Check]** Run `scripts/check_balance.py`. Expected: the matrix network's $D(t)$ and the two depth-3 differences on your formula to $10^{-9}$ relative; flow endpoints from 200 random starts with $||a_\infty| - |b_\infty|| = O(1)$ at $\gamma = 0$ (anywhere on $ab = 1$, A1) and at round-off at $\gamma = 0.1$; the per-neuron gap of the matrix network at round-off (`balance.png`).

## 3 Two layers: a continuous transition

**Problem (`stationary`): Stationary points and the bifurcation at one half** [Derive + Implement + Check, 50 min] **[you]**

(a) **[Derive]** With Problem (`balance`), find *all* stationary points of Equation 1 for $\gamma > 0$. Then compute the Hessian $H(a, b)$ of Equation 1, and its eigenvalues and eigenvectors at the origin and at each nonzero stationary point (simplify with what you know there). Classify every point for $\gamma < \frac{1}{2}$, $\gamma = \frac{1}{2}$ and $\gamma > \frac{1}{2}$, and say what happens to the nonzero points as $\gamma \uparrow \frac{1}{2}$. **Deliverable**: the stationary points as functions of $\gamma$; the learned product $p_\infty(\gamma)$ from a small balanced start; both eigenvalue pairs (type them into the `FORMULAS` block of `scripts/check_bifurcation.py`); the classification. *Hints, gentle to strong:* (1) Only $a = b$ and $a = -b$ survive (a). (2) On $a = -b$, $r = -a^2 - 1 < 0$; look at the sign of $\partial_a L_\gamma$. (3) At a nonzero point, $a^2 = b^2 = 1 - 2\gamma$ makes the diagonal of $H$ the same constant.

(b) **[Implement]** In `wdecay/stationary.py`:

`def hessian_wd(theta, gamma)` · `def stationary_points(gamma)` · `def learned_product(gamma)`

- **`hessian_wd`** $H(\theta)$ of Equation 1 for points of shape `(..., 2)`, in closed form.
- **`stationary_points`** a list of `(point, kind)`, `kind` in `"minimum"`, `"saddle"`, `"maximum"`, `"degenerate"`.
- **`learned_product`** $p_\infty(\gamma)$ from (a), broadcasting over arrays.

To test your implementation, run `uv run pytest a2-weight-decay -k test_stationary` (the adapters `run_hessian_wd`, `run_stationary_points`, `run_learned_product` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(c) **[Check]** Run `scripts/check_bifurcation.py`. It integrates Equation 2 from $(0.1, 0.1)$ to $t = 3000$ for 79 values of $\gamma$. Expected: $ab$ on your $p_\infty(\gamma)$ to $10^{-7}$, a kink (not a jump) at $\gamma = \frac{1}{2}$ with $p_\infty$ vanishing linearly (log-log slope 1), against the ridge curve of the example; autograd Hessian eigenvalues equal to yours to round-off (`bifurcation.png`).

## 3.1 Critical slowing down

Near a stable fixed point $\theta^*$, the flow is linear to first order (A0): $\theta(t) - \theta^* \approx \sum_i c_i e^{-\lambda_i t} \boldsymbol{v}_i$ with $(\lambda_i, \boldsymbol{v}_i)$ the eigenpairs of $H(\theta^*)$. At late times the slowest mode wins, so the distance to $\theta^*$ shrinks by a factor $e$ every

$$\tau = \frac{1}{\lambda_{\min}}(H(\theta^*)), \tag{3}$$

the *relaxation time*. *Critical slowing down* is the divergence of $\tau$ at a bifurcation, where some $\lambda_i \to 0$. Ang's post notes it in the margin of its phase diagram [J. M. L. Ang, 2026, §4]. Define also the *time to converge* from a start $\theta_0$,

$$t_\varepsilon = \min\{t : \|\theta(t) - \theta^*\| \le \varepsilon\}. \tag{4}$$

**Problem (`slowing`): Critical slowing down** [Derive + Implement + Check, 45 min] **[you]**

(a) **[Derive]** Show that on the line $a = b = u$ the product $p = u^2$ obeys A1's logistic equation $\dot{p} = 2p(s_{\text{eff}} - p)$ with an effective target $s_{\text{eff}}$. Find $s_{\text{eff}}(\gamma)$; it is negative for $\gamma > \frac{1}{2}$. **Deliverable**: $s_{\text{eff}}(\gamma)$.

(b) **[Derive]** From Problem (`stationary`), give $\tau(\gamma)$ by Equation 3 for every $\gamma > 0$, $\gamma \neq \frac{1}{2}$. Near the critical point $\tau \approx C_-/\left(\frac{1}{2} - \gamma\right)$ below and $C_+/\left(\gamma - \frac{1}{2}\right)$ above: find $C_-$, $C_+$ and their ratio. Where else does $\tau$ diverge, and which direction is slow there? **Deliverable**: $\tau(\gamma)$; $C_-$, $C_+$; the other divergence and its slow direction.

(c) **[Derive]** At $\gamma = \frac{1}{2}$ exactly, solve for $p(t)$ from a balanced start $p_0$. How does $\|\theta(t)\|$ decay? **Deliverable**: $p(t)$ (type it into `critical_product` in `scripts/check_slowing.py`); the power of $t$.

(d) **[Derive]** Using A1's solution of the logistic equation with target $s_{\text{eff}}$, find $t_\varepsilon$ of Equation 4 in closed form for a balanced start $a_0 = b_0 = u_0 > 0$. For fixed $\varepsilon$ and $u_0$, is the log-log slope of $t_\varepsilon$ against $|\gamma - \frac{1}{2}|$ exactly $-1$? Why or why not? **Deliverable**: $t_\varepsilon(\gamma, u_0, \varepsilon)$, valid on both sides and at $\gamma = \frac{1}{2}$; one sentence on the slope. *Hints, gentle to strong:* (1) $\|\theta - \theta^*\| = \sqrt{2}\,|u - u^*|$ on the line. (2) Solve for the $p_\varepsilon$ at which the distance equals $\varepsilon$, then invert A1's $u(t)$; the formula $t = \frac{1}{2s}\ln\frac{p(s-p_0)}{p_0(s-p)}$ holds for negative $s$ too. (3) $t_\varepsilon \approx \tau \ln(K/\varepsilon)$ where $K$ depends on $\gamma$.

(e) **[Implement]** In `wdecay/slowing.py`:

`def relaxation_time(gamma)` · `def time_to_within(gamma, u0, eps)`

- **`relaxation_time`** $\tau(\gamma)$ from (b); `np.inf` at $\gamma = \frac{1}{2}$; broadcasts.
- **`time_to_within`** $t_\varepsilon$ from (d), scalars; `0.0` if the start is already within $\varepsilon$.

To test your implementation, run `uv run pytest a2-weight-decay -k test_slowing` (the adapters `run_relaxation_time`, `run_time_to_within` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(f) **[Check]** Run `scripts/check_slowing.py`. For 9 values of $|\gamma - \frac{1}{2}|$ from $10^{-3}$ to $10^{-1}$ on each side it measures $\tau$ as the time to go from distance $10^{-4}$ to $10^{-8}$, over $\ln 10^4$. Expected: your $\tau$ within 0.1%, log-log slopes $-1.00$ on both sides, $\tau\,|\gamma - \frac{1}{2}|$ equal to your $C_-$, $C_+$; $t_\varepsilon$ on your formula to $10^{-6}$ relative, with slopes a little shallower than $-1$; at $\gamma = \frac{1}{2}$ a straight line on log-log axes with your exponent (`slowing.png`).

## 4 Depth 3: a discontinuous transition

Add a third scalar layer, $f = abcx$:

$$L_\gamma(a, b, c) = \frac{1}{2}(abc - s)^2 + \gamma(a^2 + b^2 + c^2). \tag{5}$$

Ziyin et al. prove that with two or more hidden layers the origin is always a local minimum (their Proposition 2, §4.1), that a nonzero solution exists only below a critical weight decay that depends on the architecture and the data (Proposition 3), and that common initializations can start inside the zero solution's basin (§5, Figure 3). The anchor gives all of this in closed form.

**Problem (`depth3`): The jump at depth 3** [Derive + Implement + Check + Predict, 55 min] **[you]**

(a) **[Derive]** Compute $\nabla^2 L_\gamma$ of Equation 5 at the origin. Why do the cross terms that made the two-layer origin a saddle vanish here? Expand $\frac{1}{2}(abc - s)^2$ and count degrees. **Deliverable**: the Hessian at the origin; its type for every $\gamma > 0$.

(b) **[Derive]** Using Problem (`balance`) (c), show that every nonzero stationary point has $|a| = |b| = |c| = u > 0$ and $abc > 0$, and that $u$ solves a quartic which you can write as $\gamma = g(u)$. Sketch $g$ on $u > 0$; find $\gamma^*(s) = \max_u g(u)$, the maximizer $u^*$, and the product at the maximum. Evaluate $\gamma^*(1)$ to four digits. **Deliverable**: the quartic; $\gamma^*(s)$ in closed form; $p$ just below $\gamma^*$ (type it into `product_at_jump` in `scripts/check_depth3.py`). *Hints:* (1) Multiply the three components of $\nabla L_\gamma = 0$ by $a$, $b$, $c$. (2) $g'(u) = (s - 4u^3)/2$.

(c) **[Derive]** At a balanced point $(u, u, u)$, $H$ has the form $\alpha I + \beta(\mathbf{1}\mathbf{1}^\top - I)$. Find its eigenvalues (use the quartic to eliminate $\gamma$) and say which of the two roots is a minimum and which a saddle. **Deliverable**: the eigenvalue along $(1, 1, 1)$ and the one across it; the types.

(d) **[Derive]** For which $\gamma$ does the nonzero minimum have the same loss as the origin, $s^2/2$? Between that $\gamma_c$ and $\gamma^*$, which is the global minimum? **Deliverable**: $\gamma_c(s)$ in closed form; one sentence. *Hint:* with $q = u^3$ and the quartic, the loss at a stationary point is a quadratic in $q$.

(e) **[Predict]** Before running the check, write in `notes.md`: starting from $(1, 1, 1)$, how does the learned $abc$ change as $\gamma$ grows past $\gamma^*$? If $\gamma$ is then swept back down to 0, warm-starting each run from the previous end point plus a small random kick, at which $\gamma$ does the depth-3 net start learning again? The depth-2 net?

(f) **[Implement]** In `wdecay/depth3.py`:

`def balanced_roots(gamma, s=1.0)` · `def critical_gamma(s=1.0)` · `def coexistence_gamma(s=1.0)` · `def balanced_stationary(gamma, s=1.0)`

- **balanced_roots** the positive roots $u$ of your quartic, ascending (two or none).
- **critical_gamma** $\gamma^*(s)$ from (b).
- **coexistence_gamma** $\gamma_c(s)$ from (d).
- **balanced_stationary** `[(u, kind), ...]` with `kind` from (c).

To test your implementation, run `uv run pytest a2-weight-decay -k test_depth3` (the adapters `run_balanced_roots`, `run_critical_gamma`, `run_coexistence_gamma`, `run_balanced_stationary` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(g) **[Check]** Run `scripts/check_depth3.py`. Expected: the autograd Hessian at the origin equal to your (a); from $(1, 1.01, 0.99)$ the learned *abc* on your larger root cubed to $10^{-9}$ and jumping to 0 at your $\gamma^*$, against the depth-2 product for contrast; bisection on $\gamma$ reproducing your $\gamma^*(s)$ to $10^{-5}$ relative for $s = 0.25, 0.5, 1, 2$ and a log-log slope in $s$ that you can predict from (b); a hysteresis loop at depth 3 and none at depth 2 (`depth3.png`). Compare the exponent of $s$ with Ziyin et al.'s Proposition 3 at $D = 2$.

## 4.1 Which starts fall into zero

A *basin of attraction* of a stable point is the set of starts whose flow converges to it. The boundary between two basins is made of the *stable manifolds* of saddles: the sets of starts that flow into the saddle itself.

**Problem (`basins`): Basins of the zero solution** [Derive + Implement + Check, 35 min] **[you]**

(a) **[Derive]** On the balanced ray $a = b = c = u$ (it is invariant: why?), write the 1-D flow $\dot{u} = F(u)$ for Equation 5, draw its phase line (sign of $F$ between its zeros, including $u < 0$), and state where every start $u_0 \in \mathbb{R}$ ends, for $\gamma < \gamma^*$ and $\gamma > \gamma^*$. Then find the radius $u_s$ of the zero basin on the ray to leading order in small $\gamma$. **Deliverable**: the endpoint rule; $u_s \approx$ a simple function of $\gamma$ and $s$ (type it into `small_gamma_radius` in `scripts/check_basins.py`).

(b) **[Derive]** The plane $a = b$ is also invariant. Which saddle's stable manifold separates the two basins in it, and why does the boundary in the half-plane $a = b < 0$ look like a mirror image? (Find a second symmetry of Equation 5.) **Deliverable**: two sentences.

(c) **[Implement]** In `wdecay/basins.py`:

`def balanced_endpoint(u0, gamma, s=1.0) · def zero_basin_radius(gamma, s=1.0)`

- **balanced_endpoint** $u_\infty$ for the start $(u_0, u_0, u_0)$, from (a); broadcasts over `u0`.
- **zero_basin_radius** the exact $u_s$ (`np.inf` above $\gamma^*$).

To test your implementation, run `uv run pytest a2-weight-decay -k test_basins` (the adapters `run_balanced_endpoint`, `run_zero_basin_radius` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(d) **[Check]** Run `scripts/check_basins.py`. Expected: 201 balanced starts ending where you predict (to $10^{-10}$, away from $u_s$) with the measured edge bracketing your $u_s$ at $\gamma = 0.1$; the basin map of the plane $a = b$ cut by the saddle's stable manifold; for i.i.d. $\mathcal{N}(0, \sigma^2)$ starts at $\gamma = 0.03$, the fraction that learns rising from 0 to 1 as $\sigma$ crosses a few times $u_s$, later for the weaker signal $s = 0.4$; your small-$\gamma$ radius within 1% of the exact one for $\gamma \leq 0.1$ (`basins.png`).

In `notes.md`: read off $\sigma_{1/2}$, the scale at which half the starts learn, for both $s$, and compare with $u_s$.

## 5 Finite steps: oscillation and divergence

GD with step $\eta$ on Equation 1,

$$\theta_{k+1} = \theta_k - \eta \nabla L_\gamma(\theta_k), \tag{6}$$

has the same fixed points as the flow. A0 showed that GD on a quadratic with curvature $\lambda$ converges iff $0 < \eta\lambda < 2$; near a fixed point, GD is that quadratic in each eigendirection of $H(\theta^*)$. Past the threshold the iterate first flips sign along the offending eigenvector every step; whether it then settles into a bounded oscillation or diverges is decided by the nonlinear terms. Ang's post shows the resulting diagram, with analytic stability boundaries and numerically observed oscillation and divergence regions [J. M. L. Ang, 2026, §4, the phase-diagram figure; the offline copy has only its caption].

**Problem (`finite-step`): The phase diagram of GD with weight decay** [Derive + Implement + Check + Predict, 50 min] **`[you]`**

---

(a) **`[Derive]`** Linearize Equation 6 at the fixed point the flow converges to. Using Problem (`stationary`), find $\eta_{\max}(\gamma)$ such that it is linearly stable iff $\eta < \eta_{\max}$, for $\gamma < \frac{1}{2}$ and $\gamma > \frac{1}{2}$. It has three pieces; on each, name the eigenvector that loses stability. Where is $\eta_{\max}$ largest, and what are its limits as $\gamma \to \frac{1}{2}$ from either side and as $\gamma \to 0$? **Deliverable**: $\eta_{\max}(\gamma)$; the unstable direction on each piece; the maximum and the limits.

(b) **`[Predict]`** Before running the check, write in `notes.md`: for which $\gamma$ do you expect a band of bounded oscillation above $\eta_{\max}$, and for which immediate divergence? (Think about which direction goes unstable, and what A1 said happens to $c$ when $\eta^2 r^2 > 2$.)

(c) **`[Derive]`** On the invariant line $a = b = u$ with $s_{\text{eff}} = 1 - 2\gamma > 0$, Equation 6 is the 1-D map $u \mapsto u - \eta u(u^2 - s_{\text{eff}})$. Find the $\eta$ at which its fixed point $\sqrt{s_{\text{eff}}}$ loses stability. Then substitute $v = u\sqrt{\eta/\mu}$, $\mu = 1 + \eta s_{\text{eff}}$, to get $v \mapsto \mu(v - v^3)$, and find the largest $\mu$ for which the interval $|v| \le \sqrt{1 + 1/\mu}$ is mapped into itself; beyond it almost every orbit escapes. **Deliverable**: the oscillation onset and the divergence onset as functions of $\gamma$ (type both into `scripts/check_finite_step.py`). *Hints, gentle to strong:* (1) $|\mu(v - v^3)| > |v|$ iff $v^2 > 1 + 1/\mu$. (2) On $|v| \le 1$ the map's largest value is at $v = 1/\sqrt{3}$. (3) The condition is a cubic in $\mu$ with an integer root.

(d) **`[Implement]`** In `wdecay/finite_step.py`:

`def max_stable_step(gamma)` · `def predicted_phase(eta, gamma)`

- **`max_stable_step`** $\eta_{\max}(\gamma)$ from (a); broadcasts.
- **`predicted_phase`** `"nonzero"`, `"zero"` or `"unstable"` by linear stability.

To test your implementation, run `uv run pytest a2-weight-decay -k test_finite_step` (the adapters `run_max_stable_step`, `run_predicted_phase` in `tests/adapters.py` are pre-wired to these signatures; edit them only if yours differ).

(e) **`[Check]`** Run `scripts/check_finite_step.py` (10,000 GD steps on each of $10^4$ cells, vectorized, about a second). Expected: from $(0.6, 0.4)$, your `predicted_phase` agrees with the measured converge/not-converge label on every cell more than 3% from your boundary; oscillation in only

part of the $\gamma$ range (check your prediction), with a ragged edge to the divergence region; from $(0.45, 0.45)$, both onsets within 1% (the $\eta$ grid step) of your (c) (`finite_step.png`). Compare the left panel with the post's figure caption: which of its boundaries are analytic, and which only numerical, in your derivation?

## 6 Reading

**Problem (`ziyin`): Exact solutions of a deep linear network** [Read, 20 min] **`[you]`**

Read Ziyin et al. [L. Ziyin et al., 2022] §1 (the list of implications), §3.1 (Lemma 1 and Theorem 1), §4.1 (Propositions 2 and 3) and §5 ("Insufficiency of the existing initialization schemes", "Prediction variance of stochastic nets"). Answer briefly in `notes.md`:

(a) Their hidden neurons are stochastic: $f = a\varepsilon bx$ with $\mathbb{E}[\varepsilon] = 1$, $\mathbb{E}[\varepsilon^2] = 1 + \sigma^2$ (dropout is an example). Compute $\mathbb{E}_\varepsilon\left[\frac{1}{2}(a\varepsilon b - 1)^2\right]$ and show the noise acts as an extra penalty on the product. Does it move the two-layer critical $\gamma$? What does it do to $p_\infty$? Compare with their remark in §5 ("Learnability of a neural network") that the two-layer condition does not depend on stochasticity.

(b) Explain, with your Problem (`depth3`) (a), why zero is a bad local minimum for two or more hidden layers but a saddle or the global minimum for one. Which term of the loss is responsible?

(c) Their §5 argues that initializations of radius about $1/\sqrt{\text{width}}$ can start in the zero basin when the signal $\|\mathbb{E}[xy]\|$ is weak. Which of your measurements is the scalar version of this, and how does your basin radius scale with $s$?

## 7 What to write down

In `notes.md`, three to five sentences: why weight decay balances layers; what makes the two-layer transition continuous and the three-layer one discontinuous; why relaxation slows down near a continuous transition (and what plays that role at $\gamma^*$); one question for a tutorial.

## References


[1] J. M. L. Ang, "Training Dynamics." 2026. Science of Deep Learning: Taster Series, 16 October 2026. Offline: papers/ang2026-training-dynamics-post.md.

[2] L. Ziyin *et al.*, "Exact Solutions of a Deep Linear Network." Advances in Neural Information Processing Systems 35 (NeurIPS), 2022. arXiv:2202.04777. Offline: papers/ziyin2022-exact-solutions-deep-linear-wd.pdf.